

Espera de Procesos con `wait()` y `waitpid()`

Malak Sanchez

Monitorias Sistemas Operativos

22 de septiembre de 2026

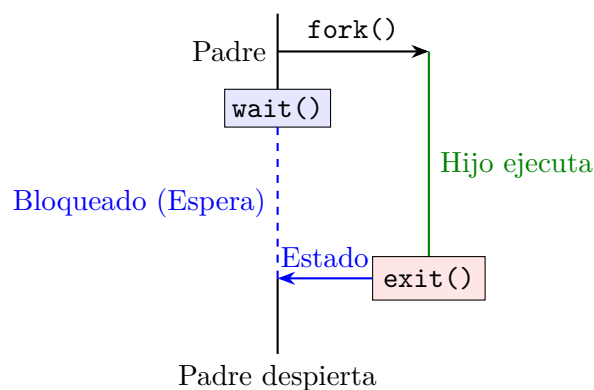
1. Motivación

Por defecto, tras un `fork()`, el padre y el hijo corren en paralelo y no sabemos quién terminará primero. Sin embargo, en la mayoría de aplicaciones, el padre necesita esperar a que el hijo termine antes de proceder (por ejemplo, para procesar los datos que el hijo generó). Sin un mecanismo de espera, el programa sería un caos asíncrono.

2. Idea Fundamental

El Sistema Operativo nos provee de las funciones `wait` y `waitpid`. Estas permiten que un proceso padre se “suspenda” (pase a estado bloqueado) hasta que uno o varios de sus hijos terminen su ejecución o cambien de estado, permitiendo además recoger su valor de salida.

3. Flujo de Espera



4. Sintaxis Básica

```
#include <sys/wait.h>

pid_t pid = wait(&status); //0 simplemente wait(NULL) si no nos
    importa el estado de salida
pid_t pid = waitpid(target_pid, &status, options);
```

5. Ejemplo Mínimo Funcional

```
1 #include <stdio.h>
2 #include <sys/wait.h>
3 #include <unistd.h>
4
5 int main() {
6     int status;
7     if (fork() == 0) {
8         printf("[HIJO] Trabajando...\n");
9         sleep(2);
10        return 42; // Estado de salida
11    } else {
12        printf("[PADRE] Esperando...\n");
13        wait(&status);
14        if (WIFEXITED(status)) {
15            printf("[PADRE] El hijo termino con: %d\n",
16                WEXITSTATUS(status));
17        }
18        return 0;
19    }
```

6. Explicación Paso a Paso

1. El padre invoca `wait(&status)`. Si el hijo aún no termina, el padre se “duerme”.
2. El hijo termina con `return 42` (o `exit(42)`).
3. El Kernel despierta al padre y coloca la información del hijo en la variable `status`.
4. **Control de estados:** Para interpretar correctamente el valor de `status`, POSIX define macros fundamentales:
 - `WIFEXITED(status)`: Verdadero si el hijo terminó normalmente.
 - `WEXITSTATUS(status)`: Retorna el código de salida real (0 – 255).
 - `WIFSIGNALED(status)`: Verdadero si el hijo murió por una señal (ej: *Segfault*).
 - `WIFSTOPPED(status)`: Verdadero si el proceso hijo se ha detenido (*Stopped*).
 - `WSTOPSIG(status)`: Retorna el número de la señal que causó que el proceso se detuviera.

7. Qué ocurre en memoria

El `wait` limpia la entrada del proceso hijo en la Tabla de Procesos (cosecha el Zombie). El valor de `status` es un entero que contiene bits específicos para el código de salida, señales de error y terminaciones forzosas.

8. Patrones comunes de uso

Esperar por un hijo específico ignorando a los demás:

```
1 waitpid(child_pid, &status, 0);
```

Esperar sin bloquearse (modo no-bloqueante):

```
1 waitpid(-1, &status, WNOHANG); // Retorna 0 si nadie ha terminado  
    aun
```

9. Errores Comunes

- **Usar `status` directamente:** El valor de `status` no es el código de salida directamente; *debe* pasarse por la macro `WEXITSTATUS`. No es necesario el uso de la macro, pero es recomendable.
- **Wait sin hijos:** Llamar a `wait` cuando no hay hijos vivos devolverá `-1`.

10. Buenas Prácticas

- Utilizar `WIFEXITED` antes de `WEXITSTATUS` para asegurar que el hijo terminó normalmente y no por un error de memoria o señal.
- Usar un bucle `while(wait(NULL) > 0)` para esperar a todos los hijos creados.

11. Ejercicios

1. Cree tres procesos hijos con tiempos de espera diferentes. Use `wait()` para que el padre informe el PID de cada hijo en el orden real en que termina.
2. Implemente un programa donde el hijo finalice mediante `abort()` e investigue qué macros de `wait` permiten detectar que terminó por una señal y cuál fue esa señal.